Andrew Centa

HW 4

CSCI 5512

4/30/26

1) Complete Value Perfect Information (VPI) $VPI(E_j)$ evaluating the value of receiving the battle observation history E_j for coop j given observation history E_i for coop i where value is # of crowns that could be received in the current tournament for an action of moving to a coop to battle.

a) show VPI is non-negative for any j and E

eqn from text:

$$VPI_e(E_j) = \left(\sum_k P(E_j = e_{jk} \mid e)\, EU(\alpha_{ejk} \mid E, E_j = e_{jk})\right) - EU(\alpha \mid e)$$

$$EU(\alpha \mid e) = \max_a \sum_s P(Result(a) = s' \mid \alpha, e)\, U(s') \quad \text{[utility of current best option]}$$

$$EU(\alpha_{ej} \mid e, e_j) = \max_a \sum_{s'} P(Result(a) = s' \mid a, e, e_j)\, U(s') \quad \text{[utility of new best option]}$$

$\therefore$

VPI = (probability of the best option * utility of best option) − (utility of current option)

intuitively, $VPI \geq 0$ because the "utility of the best option" will always be larger (or equal) than the "utility of the current option". If the best option had less utility than the current option, the entire reason to find the information would be meaningless.

$\therefore$

$$\boxed{\begin{gathered} EU(\alpha_{ejk} \mid E, E_j = e_{jk}) \geq EU(\alpha \mid E, E_j = e_{jk}) \\ \therefore \\ VPI_E(E_j) \geq 0 \end{gathered}}$$

In the chicken scenario, gaining observation history of E_j for coop j can only result in learning about more crowns or would not change the current known number. It would never make an agent lose information.

b) Assume E_j is observations of 1 battle for every pairing of 64 chickens in the 4 coop problem 3 domain, E_i is for coop i. If a chicken has ability 4 in coops i and j, what is the <u>largest VPI</u> the observation could provide? what if chicken has 3 in coop i and 4 in coop j?

<u>chicken N: $AB_i = 4$, $AB_j = 4$</u>

*VPI measured in crowns

This chicken has 2 options:

1) move to i
2) move to j

The best outcome of E_j is finding all other chickens in j have a lower score, assuring a win and a crown in j.

$$VPI_E(E_j) = \underset{1\text{ crown}}{EU(E_j)} - \underset{0\text{ crown}}{EU(E_i)}$$

(In a max VPI situation, E_j has no other 4 chicks, but E_i has only $AB_i = 4$ chicks. So moving assures a win)

$$\boxed{VPI_E(E_j) = 1 \text{ crown}}$$

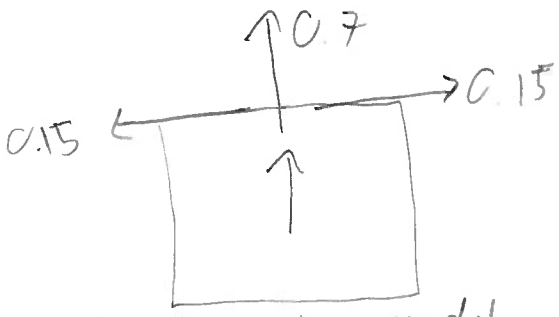
<u>chicken N: $AB_i = 3$; $AB_j = 4$</u>

This chicken will always move to coop j because they have a higher score. VPI is pointless. Observations always cost something (in chicken problem its a wasted battle in the spectator zone). The cost of getting the information is too high and unlikely to change the outcome so VPI = 0

$$VPI_E(E_j) = \underset{1\text{ crown}}{EU(E_j)} - \underset{1\text{ crown}}{EU(E_i)}$$

$$\boxed{VPI_E(E_j) = 0 \text{ crowns}}$$

2) Markov Decision Process (MDP). To move, move must be successful with prob = 0.7, P = 0.15 agent moves left, 0.15 agent moves right.



(a) transition model

(b) state utilities for S

0.747	0.816	0.874	0.931
0.690			1
0.618	0.551	0.487	-1
0.551	0.504	0.454	0.187

a) Define expected value of a state in MDP. How is general policy related to the value function?

Expected value is defined as:

$$U^{\pi}(s) = E\left[\sum_{t=0}^{\infty} \gamma^t R(S_t)\right] \quad \text{(eqn 17.2)}$$

A general policy might be:
give position of block "S"

$$U(s) = 0.7(\text{Forwards}) + 0.15(\text{Left}) + 0.15(\text{Right}s)$$

They are related in that they both find value, although the expected value includes a time element. A general policy is for any given block and is discrete.

b) Policy iteration algorithm: The algorith alternates between these 2 steps from some initial policy π_0:

1) Policy evaluation: given a policy π_i, calculate the utility of each state if π_i were executed

2) Policy improvement: calculate new Max expected utility policy (π_{i+1}) using 1 step look ahead based on U_i

The algorith terminates when a policy improvement step yields no change in utility.

$$\pi^*(s) = MEU = \arg\max \sum_{s'} P(s'|s,a)U(s')$$

Can we bound error in the policy?

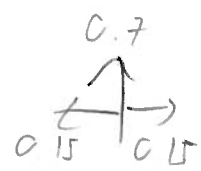
Yes. Bounding error is finding a worst-case limit on how wrong the answer can be. For policy iteration, error is tracked and plotted (figure 17.6). This implies that an error can be chosen as a bound to drive a termination point for the algorithm.

[is this $R(s)$?]

c) Find the optimal policy (π^*)

	1	2	3	4
4	.747	0.816	0.874	0.931
3	0.690			1
2	0.618	0.551	0.487	-1
1	0.551	0.504	0.454	0.187

	1	2	3	4
4	→	→	→	↓
3	↑			+1
2	↑	←	←	-1
1	↑	←	←	←



In this case, the optimal policy is usually the same as moving to the highest number, but not always. Many squares [(3,1), (4,1), (4,2), (4,3), (4,4)] only have 1 option and are the same. But some squares have multiple options. All adhere to the Bellman function π^*. For example, square (1,2) has 3 options ↑, ←, →. Their scores are 0.536, 0.544, & 0.476 respectively, so ← is the best option. This disagrees with the highest utility neighbor which claims ↑ & ← are equal. The left & right steps must be considered.

The equation is $U(s) = R(s) + \gamma \max \sum_{s'} P(s'|s,a)U(s')$

ex. $(1,2)\uparrow = 0.7(0.551) + 0.15(0.454) + 0.15(0.551) = 0.53645$

$(1,2)\leftarrow = 0.7(0.551) + 0.15(0.504) + 0.15(0.551) = \boxed{0.54395}$

$(1,2)\rightarrow = 0.7(.454) + 0.15(.504) + .15(.551) = 0.47605$

c (cont.)

Here I assess each square, but move quickly through obvious options.
R(s) omitted for comparative calcs.

(1,1) ↑ $(0.618)(0.7) + (0.504)(0.15) + (0.551)(0.15) = 0.591$

→ $(0.504)(0.7) + (0.551)(0.15) + (0.618)(0.15) = 0.528$

(1,2) ← * done on previous page

(1,3) ↑ $.487(0.7) + 0.504(.15) + .187(.15) = 0.444$

→ $.187(.7) + .487(.15) + .454(.15) = 0.27$

$\boxed{\leftarrow}$ $.504(.7) + .487(.15) + .454(.15) = \boxed{0.494}$

(1,4) ← is only reasonable, going to −1 does not make sense

(2,1) ↑ $0.69(0.7) + (0.551)(0.15) + 0.615(0.15) = 0.65$

→ $0.551(.7) + .69(0.15) + 0.551(0.15) = 0.57$

↓ $0.551(0.7) + 0.618(0.15) + 0.551(0.15) = 0.56$

(2,2) $\boxed{\leftarrow}$ $.615(.7) + .551(.15) + .504(0.15) = \boxed{0.59}$

(2,3) $\boxed{\leftarrow}$ want no chance of −1

(3,1) $\boxed{\uparrow}$ (only realistic option)

(4,1) $\boxed{\rightarrow}$ only realistic option

(4,2) $\boxed{\rightarrow}$ only realistic option

(4,3) $\boxed{\rightarrow}$ only realistic option

(4,4) $\boxed{\downarrow}$ only realistic option

So it only really differs for (1,2) in this case.

d) Construct a reward function $R(s)$ that imposes a -1 penalty when agent bumps walls.

Reward must depend on states and possible actions.

$$R(s,a) = \begin{cases} +1 & \text{if } s \text{ is at } 1 \text{ end state} \\ -1 & \text{if } s \text{ is at } -1 \text{ end state} \\ -1 & \text{if } a \text{ from } s \text{ bumps wall} \\ -0.1 & \text{otherwise (to hasten agent to a finish point)} \end{cases}$$

---

e) Per the "Execute_Temporal_Diff_Learning.py"

cost: -0.1

>	>	>	+1	3
^		^	-1	2
^	<	^	^	1
1	2	3	4	

cost: -0.04

>	>	>	+1	3
^		^	-1	2
^	<	^	^	1
1	2	3	4	

cost: -0.08

>	>	^	+1	3
^		^	-1	2
^	<	^	<	1
1	2	3	4	

cost -0.02

>	>	^	+1	3
^		^	-1	2
^	<	^	<	1
1	2	3	4	

cost -0.001

^	<	^	+1	3
^		^	-1	2
<	<	<	v	1
1	2	3	4	

When the cost is higher, the agent is more interested in finishing as quickly as possible; even if it means taking a riskier path. When the cost is less, the agent is OK with taking a longer path if there's a better chance of a good result. This can be seen with tile (3,3). Larger step costs will move directly towards +1 even though there is a 15% chance they could end up by -1. Lower cost models move up here. It is only a 15% of +1, but 0% of getting closer to -1. Similarly, (3,4) shows that costly models will go to -1 on purpose because +1 is so far away.

f) if agent has energy budget, the energy must be accounted for in the state representation and the reward or transition structure.

Energy must be tracked in the state representation so that each move reduces "fuel" by x. Like how google maps considers both time & fuel economy. This complicates the state by adding a third variable.

If energy is ever 0, this must trigger a terminal rule. It should be attached to a large negative number so that running out of fuel is heavily disincentivized.

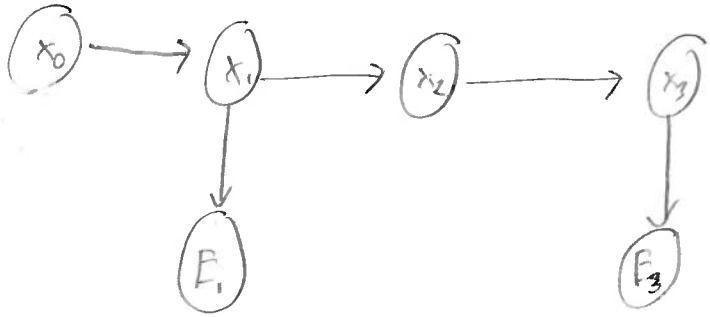
The reward function may look like

$$R(s, a, e) = \begin{cases} +1 & \text{if at 1 end state} \\ -1 & \text{if at -1 end state} \\ -1 & \text{if wall bumped} \\ -1 & \text{if energy} = 0 \\ -0.5 & \text{otherwise} \end{cases}$$

The gamma is made larger to hasten the agent. The reward function needs to track that running out of energy is a negative, but ~~a~~ gamma already incentivizes the agent to finish in fewest moves so increasing that accomplishes the task of using less energy without over complicating the reward function.

To implement non-terminal states which increase energy, place these tiles in the state space. The agent is incentived to move to these states since the reward function is built to make running out of energy disincentivized.

3)



$P(X_{t+1}=T \mid X_t=T)=0.75$

$P(X_{t+1}=T \mid X_t=F)=0.25$

$P(E_t=T \mid X_t=T)=0.8$

$P(E_t=T \mid X_t=F)=0.4$

$E_1=T, \quad E_3=F \quad P(X_0=T)=0.2$

a) Compute $P(X_2=T \mid E_1=T, E_3=F)$

if $E_1=T$

$P(x_0)$	
T	0.2
F	0.8

$P(x_0)$	$P(x_1)$ T	$P(x_1)$ F
T	0.75	0.25
F	0.25	0.75

$P(x_1)$	$P(x_2)$ T	$P(x_2)$ F
T	.75	.25
F	.25	.75

$P(x_2)$	$P(x_3)$ T	$P(x_3)$ F
T	.75	.25
F	.25	.75

Prediction

smoothing

[Part b]

$P(x_1)$	$P(E_1)$ T	$P(E_1)$ F
T	0.8	0.2
F	0.4	0.6

E_1

$P(x_2)$	$P(E_2)$ T	$P(E_2)$ F
T	0.8	0.2
F	0.4	0.6

E_2

$P(x_3)$	$P(E_3)$ T	$P(E_3)$ F
T	0.8	0.2
F	0.4	0.6

E_3

$$\sum_{x_0,x_2,x_3} P(x_2=T \mid x_1)\, P(E_1=T \mid x_1)\, P(E_3=F \mid x_3)\, P(x_0)\, P(x_1 \mid x_0)$$

Prediction

i) Priors for x_1: $P(x_1=T)=P(x_1 \mid x_0=T)(P x_0=T)+P(x_1 \mid x_0=F)\,P(x_0=F)$

$P(x_1=T)=(0.75)(0.2)+(0.25)(0.8)=\underline{0.35}$

Normalize: $P(x_1=F)=P(x_1=F \mid x_0=T)(P x_0=T)+P(x_1=F \mid x_0=F)\,P(x_0=F)$

$(0.25)(0.2)+(0.75)(0.8)=\underline{0.65}$

ii) Update priors with E_1

$P(X_1=T, E_1=T) = P(E_1=T \mid X_1=T)P(X_1=T) = (0.8)(0.35) = 0.28$

For normalize

$P(X_1=F, E_1=T) = P(E_1=T \mid X_1=F)P(X_1=F) = (0.4)(0.65) = 0.26$

$= \alpha \langle .28, 0.26 \rangle$

$= \langle .5185, 0.4815 \rangle$

3) Find $P(X_2 \mid E_1) = \sum_{x_1} P(X_2 \mid x_1) P(x_1 \mid E_1) = P(X_2=T \mid x_1=T) P(x_1=T \mid E_1=T) + P(x_2=T \mid x_1=F) \cdot P(x_1=T \mid E=F)$

$(0.75)(.5185) + (0.25)(.4815) = 0.50925$

$P(X_2 \mid E_1) = \langle 0.5093, 0.4907 \rangle$

Backward Pass aka smoothing

$P(-E_3 \mid X_2) = \sum_{x_3} P(-E_3 \mid x_3) P(x_3 \mid X_2) = P(-E_3 \mid x_3=T) P(x_3 \mid X_2=T) + P(-E_3 \mid x_3=F) P(x_3 \mid X_2=F)$

$(0.2)(0.75) + (0.6)(0.25) = 0.3$

For normalize $\sum P(-E_3 \mid x_3) P(x_3 \mid X_2=F) + P(-E_3 \mid x_3) P(x_3=F \mid X_2=F)$

$(0.2)(0.25) + (0.6)(0.75 = 0.5$

$P(E_3 \mid X_2) = \langle 0.3, 0.5 \rangle$

Combine

$P(X_2 \mid E_1, E_3) = \alpha P(X_2 \mid E_1) \times P(E_3 \mid X_2)$

$= \alpha \langle 0.50925 \times 0.3, 0.4907 \times 0.5 \rangle$

$= \alpha \langle 0.153, 0.245 \rangle$

$= \frac{0.153}{.153 + .245} = 0.384$

$\boxed{P(X_2 \mid E_1, E_3) = 0.384}$

b) if $E_2 = T$, Find $P(X_2 = T \mid E_1 = T, E_2 = T, E_3 = F)$

$P(X_2 \mid E_1, E_2, E_3) \propto P(E_2 \mid X_2) \cdot P(X_2 \mid E_1, E_3)$

$P(E_2 = T \mid X_2 = T) \cdot P(X_2 = T \mid E_1, E_3)$

$(0.8) \cdot (0.384) = 0.3072$

$P(E_2 = T \mid X_2 = F) \cdot P(X_2 = F \mid E_1, E_2)$

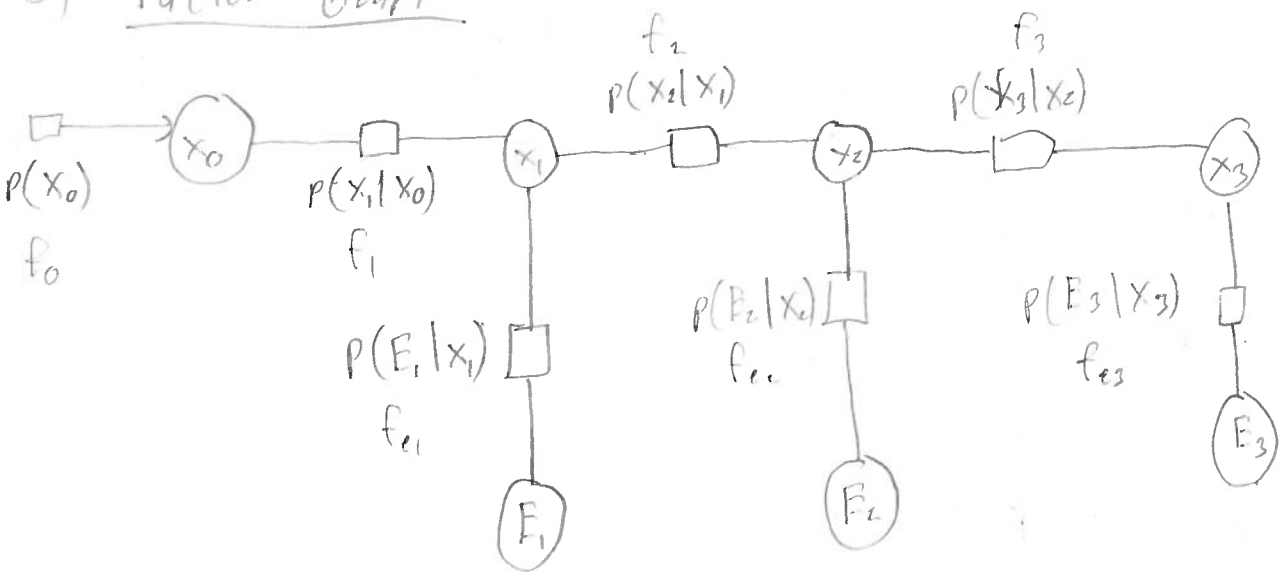
$(0.4) \cdot (1 - 0.384) = 0.24652$

$P(X_2 \mid E_1, E_2, E_3) = \alpha \langle 0.3072, 0.24652 \rangle$

$$\frac{0.3072}{0.3072 + 0.24652}$$

$$\boxed{P(X_2 \mid E_1, E_2, E_3) = 0.5548}$$

c) Factor Graph



Sum - Product Formula:

$$\mu_{f\to x}(x) = \sum_{\sim x} f(\text{neighbors of } f) \prod_{y \in ne(f)} \mu_{y\to f}(y)$$

* product of all incoming messages to the factor summed over all variables except x

example: Find equation for β_2 specifically.

$$\mu_{x_3 \to f_3}(x_3) = f_{e3}(x_3) = P(E_3|X_3)$$

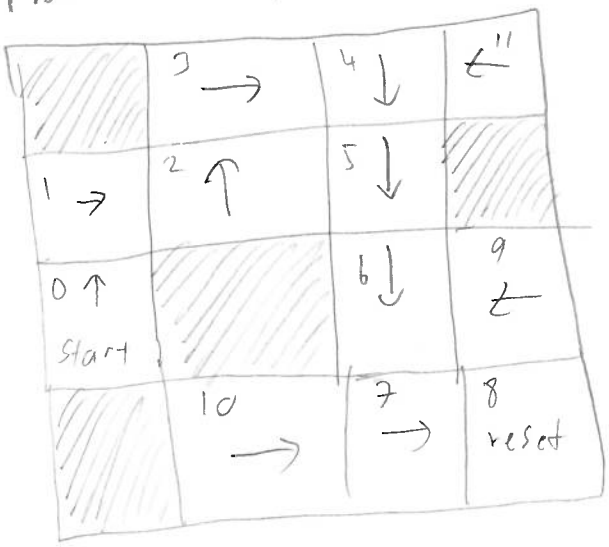
$$\mu_{f_3 \to x_2}(x_2) = \sum_{x_3} f_3(x_2, x_3) \cdot \mu_{x_3 \to f_3}(x_3)$$

$$\beta_2(x_2) = \sum_{x_3} P(x_3|x_2)\, P(E_3|x_3)$$

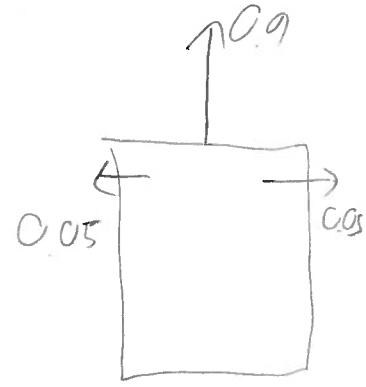
Generalize

$$\boxed{\beta_{k-1}(x_{k-1}) = \sum_{x_k} P(x_k|x_{k-1})\, P(E_k|x_k)\, \beta_k(x_k)}$$

Problem 2 (4) agent can't go on black squares



Policy



transition model

- colors are observations ; $s = \{0, 1, \ldots 11\}$ location;

a) Get policy & transition probability matrices

Policy matrix

$$
\begin{array}{c|cccccccccccc}
 & \uparrow & \rightarrow & \uparrow & \rightarrow & \downarrow & \downarrow & \downarrow & \rightarrow & \uparrow & \leftarrow & \rightarrow & \leftarrow \\
\hline
\uparrow 0 & 0.9 & 0.05 & 0.9 & 0.05 & 0 & 0 & 0 & 0.05 & 0.9 & 0.05 & 0.05 & 0.05 \\
\rightarrow 1 & 0.05 & 0.9 & 0.05 & 0.9 & 0.05 & 0.05 & 0.05 & 0.9 & 0.05 & 0 & 0.9 & 0 \\
\leftarrow 2 & 0.05 & 0 & 0.05 & 0.05 & 0.05 & 0.05 & 0.05 & 0 & 0.05 & 0.9 & 0 & 0.9 \\
\downarrow 3 & 0 & 0.05 & 0 & 0 & 0.9 & 0.9 & 0.9 & 0.05 & 0 & 0.05 & 0.05 & 0.05 \\
 & 0 & 1 & 2 & 3 & 4 & 5 & 6 & 7 & 8 & 9 & 10 & 11
\end{array}
= P(a|s)
$$

(arbitrary)

up
right
left
down

transition matrix $T(s'|s,a)$

state 0

↑ 1
→ 0
← 0
↓ 0

↑ direction

↖ destination

state 1

↑ 1 (wall)
→ 2
← 1 (wall)
↓ 0

state 2

↑ 3
→ 5
← 1
↓ 2 (wall)

state 3

↑ 3 (wall)
→ 4
← 3 (wall)
↓ 2

state 4

↑ 4 (wall)
→ 11
← 3
↓ 5

state 5

↑ 4
→ 5 (wall)
← 2
↓ 6

state 6

↑ 5
→ 9
← 6 (wall)
↓ 7

state 7

↑ 6
→ 8
← 10
↓ 7 (wall)

state 8

↑ 0
→ 0
← 0
↓ 0

state 9

↑ 9 (wall)
→ 9 (wall)
← 6
↓ 7

state 10

↑ 10 (wall)
→ 7
← 10 (wall)
↓ 10 (wall)

state 11

↑ 11 (wall)
→ 11 (wall)
← 4
↓ 11 (wall)

compiled on next page

b) Get dynamics function: $P(s'|s)$ & observation function: $P(y, a|s)$

$$P(s'|s) = \sum_a T(s'|s,a)\,P(a|s)$$

Matrix multiplication completed in python

$$P(s'|s) = \begin{bmatrix}
0.1 & 0.9 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\
0.05 & 0.05 & 0.9 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\
0 & 0.05 & 0 & 0.9 & 0 & 0.05 & 0 & 0 & 0 & 0 & 0 & 0 \\
0 & 0 & 0 & 0.1 & 0.9 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\
0 & 0 & 0 & 0.05 & 0 & 0.9 & 0 & 0 & 0 & 0 & 0 & 0.05 \\
0 & 0 & 0.05 & 0 & 0 & 0.05 & 0.9 & 0 & 0 & 0 & 0 & 0 \\
0 & 0 & 0 & 0 & 0 & 0 & 0.05 & 0.9 & 0 & 0.05 & 0 & 0 \\
0 & 0 & 0 & 0 & 0 & 0 & 0.05 & 0.05 & 0.9 & 0 & 0 & 0 \\
1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\
0 & 0 & 0 & 0 & 0 & 0 & 0.9 & 0.05 & 0 & 0.05 & 0 & 0 \\
0 & 0 & 0 & 0 & 0 & 0 & 0 & 0.9 & 0 & 0 & 0.1 & 0 \\
0 & 0 & 0 & 0 & 0 & 0.9 & 0 & 0 & 0 & 0 & 0 & 0.1
\end{bmatrix} = P(s'|s)$$

SEE Attached sheet for python printout of $P(s'|s)$

$P(y, a|s) = P(a|s)\,P(y|s)$

$$P(y|s) = \begin{array}{c} \\ R \\ Y \\ G \\ B \\ \\ \end{array} \begin{array}{c} \begin{array}{cccccccccccc} R & G & B & R & Y & R & G & Y & B & Y & G & B \end{array} \\ \begin{bmatrix} 0.9 & j & j & 0.9 & j & 0.9 & j & j & j & j & j & j \\ j & j & j & j & 0.9 & j & j & 0.9 & j & 0.9 & j & j \\ j & 0.9 & j & j & j & j & 0.9 & j & j & j & 0.9 & j \\ j & j & 0.9 & j & j & j & j & j & 0.9 & j & j & 0.9 \end{bmatrix} \\ \begin{array}{cccccccccccc} 0 & 1 & 2 & 3 & 4 & 5 & 6 & 7 & 8 & 9 & 10 & 11 \end{array} \end{array}$$

$$j = \frac{0.1}{3}$$

* Matrix multiplication in python

---

part c)

* see Question_4_part_c.py

---

part d)

* Example pseudocode link did not work *

1) Initialize with random variables

T = transition matrix

E = emission matrix

π = initial state

2) Forward procedure: Find probability of state i given previous knowledge

3) Backward Procedure: Find probability of future observations given state i

4) compute state probabilities

5) compute transition probabilities

6) update parameters

7) check convergence